Field Hours

CSCE 612 Applied Networks and Distributed Processing

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 40 minutes

Delay, bandwidth, and layers

A network moves packets, not files. A packet has a header the switches understand and a payload they usually do not. The time to get one packet from one machine to the next is four pieces added together: processing in the node, queueing behind other packets, transmission as the bits go onto the link, and propagation as the signal crosses the distance. People mix up transmission and propagation. Transmission depends on packet size and link rate. Propagation depends on distance and the speed of the signal, which is not the link rate.

Bandwidth is how many bits per second the link can accept. Delay is how long one bit, or one packet, takes. A long fat link can have huge bandwidth and still take milliseconds for the first bit to arrive. A protocol that waits for an acknowledgement before sending the next small packet will not fill that link. The later lecture on windows is the repair. This lecture is the measurement that shows the repair is needed.

Layers are a design rule. The link layer moves a frame across one hop and worries about that hop's errors. The network layer names hosts and forwards packets across hops. The transport layer recovers an end-to-end byte stream or a message, on top of a network that drops things. The application speaks to the transport. A layer that reaches into another layer's header for a shortcut will break when that header changes. The rule is not purity. It is a place to put each failure.

Protocol analysis starts with a failure, not a feature list. Ask which layer is allowed to drop, reorder, or duplicate, and which layer must hide that. If the network may duplicate packets, the transport must tolerate a replay or you will apply a write twice. Write that sentence into the design before you write code.

Each layer adds a header and a promise. The promise is the only part the layer above may rely on.

Application: a request

Transport: an end-to-end stream

Network: a hop-by-hop packet

Link: one hop's frame

One packet, one link

1. The packet is 1000 bytes, which is 8000 bits. The link is 10 million bits per second. Transmission is 8000 / 10 million = 0.8 milliseconds.

2. The link is 200 kilometers of fiber. The signal is about 200,000 kilometers per second. Propagation is 200 / 200,000 seconds = 1 millisecond.

3. Ignore queueing and processing and the packet's last bit arrives about 1.8 milliseconds after transmission starts. A smaller packet does not shrink the 1 millisecond.

Problem 1. The same fiber, and a 100-byte packet. What are transmission and propagation?

Answer. 100 bytes is 800 bits. Transmission is 800 / 10 million = 0.08 milliseconds. Propagation stays 1 millisecond. The packet is now almost pure propagation delay.

Problem 2. A link-layer protocol promises no losses on one hop. The path has three hops. May the transport assume the packet arrives?

Answer. No. Each hop can deliver to the next and the next router can still drop the packet for lack of queue space. End-to-end arrival is a transport problem unless every hop, and every queue, made a promise you actually have.

Lecture 2 · 40 minutes

Forwarding

A link delivers a frame to the other end of one wire or one wireless hop. A switch learns which port a link-layer address lives on by watching source addresses, and it forwards a frame out that port. If it has not learned the address, it floods. Flooding is correct and expensive. A switch does not understand network-layer addresses. Two switches can also loop if someone plugs a cycle, and then a flooded frame can circulate forever. Spanning-tree style blocking of ports is how Ethernet-style networks refuse the cycle. You should be able to say why a loop is fatal for a flood.

The network layer gives hosts addresses that do not change when the packet crosses a link. A router looks at the destination address and picks a next hop from a forwarding table. The longest matching prefix wins, so a specific route can override a general one. Forwarding is local and fast. Routing, the next lecture, is how the table gets built.

A packet's time to live, or hop limit, is decremented at each router. At zero the packet is dropped. That is the repair for a routing loop, which would otherwise circulate until the links melted. A traceroute is a deliberate use of that death: send packets with increasing limits and collect the ICMP-style notices from the routers that dropped them. The path you see is the path those packets took, not a promise about the next packet.

Programming this layer means reading and writing headers without assuming the payload is text. Checksums cover the header so a corrupted address is not forwarded into someone else's machine. A checksum that excludes the field you are about to change, such as the hop limit, has to be recomputed or adjusted. Forget that and honest packets look corrupt two hops later.

Longest prefix, not first match. A more specific route beats a shorter one.

1. Read the destination

2. Find every matching prefix

3. Keep the longest

4. Send to that next hop

Two routes

1. The table says 10.0.0.0/8 goes to port A, and 10.1.0.0/16 goes to port B.

2. Destination 10.1.2.3 matches both. The 16-bit prefix is longer, so the packet goes to port B.

3. Destination 10.2.2.3 matches only the 8-bit prefix and goes to port A.

Problem 1. A switch has never seen the destination link address. What does it do, and why is a cycle of switches dangerous?

Answer. It floods the frame out every other port. In a cycle the flooded copy returns and is flooded again. Without a lifetime or a blocked port, the frame multiplies. That is why the topology that floods must be a tree, or the frame must die.

Problem 2. A router decrements the hop limit and forwards. The header checksum still covers the old hop limit. What does the next router do?

Answer. It treats the header as corrupt and drops the packet. The sender of a changed header field must fix the checksum. The drop is correct behavior, not a flaky link.

Lecture 3 · 40 minutes

Routing

Routing builds the forwarding table. Distance-vector routing has each router advertise the cost to each destination to its neighbors. A router sets its cost to the minimum, over neighbors, of that neighbor's cost plus the link cost. The information is only a distance, not a path, so a router cannot see a loop in the advertisement. When a link dies, a router can count to infinity: it believes a neighbor's route that actually runs back through itself. A split horizon, or a hold-down, is a patch. The failure mode is the lesson.

Link-state routing floods the map of links, not the distances. Every router computes shortest paths on the same map, usually by Dijkstra. A loop in the forwarding tables is less natural, because everyone has the topology, and it is still possible while the flood is incomplete. The cost is the flood and the computation. The gain is that a dead link is a fact in the map, not a rumor about a distance.

Work a four-router example by hand for both algorithms or you will confuse the advertisements. Distance vector sends a vector. Link state sends each router's own links. The forwarding table at the end can match. The messages on the wire do not.

Policy is why the public Internet is not one Dijkstra. A network may refuse to carry a neighbor's route, or prefer a longer route that stays inside a contract. Policy is not a shortest path. If your design assumes every router minimizes delay, a real internetwork will violate the assumption on purpose.

Same goal, a next hop. The message each algorithm is willing to send is different.

Distance vector

Advertise a distance

No map of the path

Count-to-infinity risk

Link state

Flood each link

Everyone runs Dijkstra

Cost is the flood

Three routers in a line

1. Routers A-B-C. Each link costs 1. A advertises destination A at cost 0.

2. B sets A at cost 1 and tells C. C sets A at cost 2. Forwarding: C sends to B, B sends to A.

3. If the A-B link dies and B still hears C advertise A at cost 2, B may set A at cost 3 via C, and C will then set cost 4 via B. That is counting up, and the route is a loop.

Problem 1. In that failure, what does split horizon stop B from doing?

Answer. C learned A from B, so split horizon says C should not advertise A back to B. B then does not hear a fake route to A through C. The count does not start. Split horizon does not solve every topology. It solves this echo.

Problem 2. Link state has already flooded the dead A-B link to B and C. What next hop does C choose for A?

Answer. None. The map no longer contains a path. C withdraws the route. It does not keep a stale distance, because the input was the link, not a rumor.

Lecture 4 · 45 minutes

Reliable transport

The network drops, reorders, and duplicates. A reliable transport delivers each byte once, in order, to the application. Sequence numbers are how a receiver notices a duplicate and how it knows there is a hole. An acknowledgement says which bytes have arrived. A retransmission timer fires when an acknowledgement does not. Set the timer below the round trip and you will retransmit data that was merely slow. Set it far above and a loss sits there. Measuring the round trip, and adding a margin for variation, is the design.

Stop-and-wait sends one packet and waits. On a link with a 100-millisecond round trip and a 1-millisecond transmission, the sender is idle 99 percent of the time. A sliding window lets the sender have many unacknowledged packets in flight, up to the window. The amount of data in flight should be about the bandwidth times the round trip if you want a full link. That product is the number you compute before you pick a window size.

Cumulative acknowledgements are simple and stall when one packet in the middle is lost: the sender may retransmit everything after the hole, or, with a better design, only the hole. Selective acknowledgements name the holes. The programming exercise is a state machine, not a loop. States are the next sequence to send, the next sequence expected, and the buffer of out-of-order data. A packet that arrives twice must not be delivered twice.

Flow control is a different window, advertised by the receiver because its buffer is finite. Congestion control, next lecture, shrinks the window because the path is finite. Mixing them up means you will open the window into a full receiver or into a full router and call it the other bug.

A window of four. The sender may fill these sequence numbers before the first acknowledgement returns.

Seq 1 -> Seq 2 -> Seq 3 -> Seq 4

How much data should be in flight

1. The path is 10 million bits per second. The round trip is 100 milliseconds, which is 0.1 seconds.

2. Bandwidth times delay is 10 million times 0.1 = 1 million bits, or 125 kilobytes.

3. Stop-and-wait with a 1000-byte packet keeps 8000 bits in flight, far under a million. The window needs on the order of 125 such packets to fill the path.

Problem 1. A duplicate of sequence 5 arrives after 5 was already delivered. What does the receiver do?

Answer. It discards the payload and does not deliver it again. It may repeat the acknowledgement so the sender can move on. Delivering twice is a broken reliability layer.

Problem 2. Round trip doubles and the window in bytes stays fixed. What happens to throughput?

Answer. The path can now hold twice as much data, and the sender still fills only the old amount. Throughput falls to about half, because the sender goes idle waiting for acknowledgements. The window has to track the bandwidth-delay product.

Lecture 5 · 40 minutes

Congestion, and what the application may assume

Congestion is routers whose queues overflow. The honest signals are loss and delay. A sender that treats every loss as a reason to try harder will fill the queue again. Additive increase and multiplicative decrease is the classic response: grow the window slowly while nothing is lost, and cut it sharply on a loss. The cut is what saves the path. The slow growth is what searches for the new rate. A sender that only increases is a bug.

Fairness is not automatic. Two senders on one link, both using the same increase and decrease, tend toward equal rates because the one with the larger window loses more often and cuts more. A sender that does not cut can take the link from the ones that do. That is why a congestion algorithm is a social rule as much as a throughput rule. Your transport either participates or it is the load everyone else backs off from.

The application may assume what the transport promised and no more. A reliable ordered byte stream does not mean a message boundary. A write of 100 bytes and a write of 100 bytes can arrive as one read of 200, or as several smaller reads. If you needed records, you must frame them. A datagram transport may drop and reorder, and the application that wanted a file has to do lecture four itself or pick a different transport.

Distributed processing sits on these promises. A remote call that retries after a timeout can run twice if the first attempt was slow rather than lost. Make the operation safe to repeat, or give it an identifier the server remembers. The network will not do that for you. The application protocol is part of the course, and the retry rule is the part that corrupts data when it is wrong.

One loss. The window is cut, then it creeps back. Skipping the cut is how you stay congested.

1. Window grows by a little

2. A loss or a mark

3. Window is cut

4. Growth starts again

Two senders, one link

1. The link holds about 20 packets. Sender A has a window of 16, sender B a window of 4.

2. A loss hits. Both cut in half: A to 8, B to 2.

3. They increase one packet at a time. A is still larger, and it will be the one to cause the next loss more often, so the cuts pull them toward each other. This is a story, not a proof, and it fails if A refuses to cut.

Problem 1. Your application writes two 50-byte messages on a byte stream and the peer reads 100 bytes. Is the transport broken?

Answer. No. A byte stream does not preserve write boundaries. The reader must know the framing, for example a length prefix, to recover the two messages.

Problem 2. A remote debit times out and the client sends it again. The server applied it once and the response was lost. What balance do you get if the debit is not idempotent?

Answer. The account is charged twice. The repair is a unique request id stored at the server so the retry returns the first result and does not apply the debit again.

Lecture 6 · 180 minutes

Names and requests

An application does not send packets to a path. It sends a request to a name. DNS maps the name to an address. The mapping is cached, and a cache can be stale for as long as the time-to-live it stored. A failure that lasts exactly that long is often a DNS failure, not a routing failure.

HTTP is a request and a response over a reliable byte stream. The version you should be able to read by hand has a method, a path, headers, and an optional body. The status code is the contract. A 200 with an error inside the body is a broken contract. A 404 is a successful delivery of a refusal.

The reliable byte stream under HTTP does not know that HTTP exists. It delivers bytes in order, or it fails. Framing those bytes into requests is the application's job. If you lose the framing, you will parse the next request as the rest of the previous body.

Timeouts belong at this layer too. A request that waits forever for a server that has gone is not reliable. It is stuck. Choose a timeout and say what the user sees when it fires.

From a name to a response.

Resolve the name -> Open a byte stream -> Send a framed request -> Read a status and a body

One stale cache

1. A record says the address is 10.0.0.1 for 300 seconds. The host moves to 10.0.0.2 at second 10.

2. Resolvers that cached the record keep using 10.0.0.1 until the 300 seconds end. The routing table can be perfect and the connection still fails.

3. The fix is a shorter time-to-live, or an explicit flush, not a new route. Identify the layer before you debug it.

Problem 1. A client reads bytes until the connection closes and treats them as one HTTP response. What goes wrong if the server keeps the connection open for a second request?

Answer. The client blocks until the timeout or the close, and it may glue two responses together or wait forever. Framing uses the content length or chunking, not the close. Persistent connections require that framing.

Problem 2. Why is a 200 response with a body that says 'error' a contract bug?

Answer. Intermediaries and clients branch on the status. A 200 will be cached and treated as success. The error belongs in a 4xx or 5xx code. The body can explain it. The body cannot replace it.

Lecture 7 · 180 minutes

Congestion control, numerically

The earlier lecture said two senders share a link. This week is the rule TCP uses. Additive increase, multiplicative decrease: each round-trip without loss adds one segment to the congestion window, and a loss cuts the window in half. The window is the number of unacknowledged bytes the sender may have in flight.

Slow start doubles the window every round-trip until a threshold, then additive increase takes over. It is how a new connection finds a rate without being told the link speed. It overshoots, loses a packet, and cuts back. That loss is the probe, not necessarily a failure of the path.

Fairness in this model means two flows that share a bottleneck and use the same rule converge toward equal windows. A flow that does not cut on loss takes the link. That is why a UDP sender that ignores loss is not 'more efficient TCP.' It is a flow that refused the rule the others are using.

Bufferbloat is a large queue that delays the loss signal. The window grows until the buffer fills, and the delay rises long before the throughput does. A smaller buffer, or an explicit signal, tells the sender sooner. Capacity and delay are different axes.

One cycle of AIMD.

Window 8, a loss

Window becomes 4

Four quiet round-trips add 1 each

Window returns to 8 and probes again

Two flows, one link

1. The link carries 16 segments per round-trip. Flow A has window 12, flow B has window 4. Both see loss when the sum exceeds 16.

2. Both cut in half: A becomes 6, B becomes 2. Sum is 8, under the link.

3. Each additive increase raises both by 1 per round-trip. They meet near 8 and 8. The larger flow cut more in absolute segments. That is the fairness argument, and it assumes both flows obey the cut.

Problem 1. Flow B never reduces its window on loss. What happens to flow A if A obeys AIMD?

Answer. A keeps cutting and B does not, so B occupies the link and A's window ratchets toward zero. AIMD fairness requires both sides to decrease. A polite flow against a greedy flow loses.

Problem 2. The buffer holds 1000 packets and the bandwidth-delay product is 10 packets. Why can delay be large while throughput looks fine?

Answer. The sender fills the buffer. Throughput stays at the link rate, and each packet waits behind hundreds of others. The user sees delay. A loss-only signal arrives only after the buffer is full. The queue, not the link, is the latency.

Lecture 8 · 180 minutes

Routing protocols

Distance-vector routing exchanges, with each neighbor, the cost to every destination. A router sets its cost to a destination to the minimum, over neighbors, of the link cost plus the neighbor's advertised cost. The information is only a distance. The path is implicit. Count-to-infinity is the failure: when a link dies, routers that still point at each other increment the distance forever, or until a cap.

Split horizon and a maximum distance reduce that failure and do not make it impossible in every topology. Link-state routing floods the map. Each router computes shortest paths on the same graph. A failed link is a flooded fact, not a rumor about a distance. The cost is the flood and the requirement that the maps converge to one picture.

The earlier shortest-path lecture is the algorithm inside link state. The protocol is how the inputs arrive and how you avoid acting on a partial flood. A router that runs Dijkstra on a half-updated map will black-hole traffic until the rest of the flood arrives.

Policy is not shortest paths. An organization may refuse to carry traffic that is not its customer, even when it has a cheap path. BGP exists because the Internet is a set of policies. Pretending every route is Dijkstra will not explain a path that leaves the continent to reach a neighbor.

Two ways to learn a route.

Distance vector

Advertise distances

Count-to-infinity

Small messages

Link state

Flood the links

Each router runs shortest paths

One shared map

Three routers in a line

1. A-B-C, each link cost 1. C's route to A is through B, cost 2. B's route to A is the direct link, cost 1.

2. The link A-B fails. B should set its cost to A to infinity. If B's update is slow, C still advertises cost 2, and B may adopt C as the next hop, cost 3.

3. C then hears B's new cost and raises its own. They count upward. A maximum distance stops the count and still leaves a window where packets loop.

Problem 1. Why does link state not suffer the same count-to-infinity on this failure?

Answer. B floods 'link A-B is down.' C and B compute on a map that does not contain the link. Neither adopts a route that uses it. There is no mutually referential distance left to increment.

Problem 2. A shortest path exists through a network that refuses to transit your traffic. Why is the packet not sent that way?

Answer. The refusal is policy. The protocol that implements policy will not advertise or will not select that path. Shortest path on a graph that ignores policy answers a problem the network is not solving.

Lecture 9 · 180 minutes

Queues

A switch or a router is a queue in front of a link. When the arrival rate exceeds the service rate, the queue grows. A drop-tail queue drops the arrival that does not fit. The sender learns about congestion from the loss, one round-trip later, and every flow sees the loss together. That synchronization is a reason queues are not only storage.

Fair queuing gives each flow a share of the link instead of a share of the buffer slots. A flow that sends too fast fills its own queue and loses its own packets. The others continue. FIFO cannot do that. One fast flow occupies the FIFO queue.

Priority queues serve one class first. They are how a latency-sensitive flow skips a bulk transfer, and they are how a misconfigured high class starves everyone else. Priority without a rate limit on the high class is a starvation bug.

The end-to-end argument still holds. The queue can drop and can schedule. It cannot know the application's retransmission semantics. Reliability remains the endpoint's job. The queue's job is to allocate the link you actually have.

One fast flow, one slow flow.

FIFO

One shared buffer

The fast flow fills it

The slow flow sees the drops

Fair queuing

A queue per flow

The fast flow drops its own

The slow flow keeps its share

A link of 10, a sender of 30

1. The link serves 10 units per second. One sender offers 30, another offers 5.

2. A deep FIFO fills with the fast sender. The slow sender's packets wait behind a buffer of traffic that is not its own, or are dropped when the buffer is full of the fast sender.

3. Fair queuing serves them in proportion, or equally if you configured equal weights, so the slow sender's 5 get through and the fast sender's excess is the traffic that waits or drops.

Problem 1. Why can a larger buffer increase delay without increasing the throughput of a saturated link?

Answer. The link is already full. Extra buffer holds extra packets. Throughput stays at the service rate. Delay grows with the queue length. This is the same phenomenon as bufferbloat, seen from the queue rather than from the window.

Problem 2. A priority class has no rate cap and a buggy sender marks everything as high. What happens to the low class?

Answer. It starves as long as the high class has a packet. Priority is a preference, and without a cap it is a monopoly. Cap the high class or it is not a priority configuration you can defend.

Lecture 10 · 180 minutes

Reliable delivery

The earlier lecture sized a window. This week is the protocol that uses it. A sender numbers bytes, holds a copy until they are acknowledged, and retransmits on a timeout or on duplicate acknowledgements. A receiver accepts in-order bytes and either discards or buffers the rest. The choice changes performance, not correctness, if the sender retransmits.

A cumulative acknowledgement says every byte up to N has arrived. It is compact, and it cannot name a hole past N. Selective acknowledgements name the holes. Without them a loss at the front of a window can retransmit bytes the receiver already has.

The timeout must exceed a reasonable round-trip or you retransmit data that was only slow. An estimate of the round-trip, plus a variance term, is the usual rule. A fixed timeout chosen on a laptop will be wrong on a path that takes longer.

Idempotence is the application's problem once the byte stream is reliable. The stream may deliver a request once and the response may be lost, so the application retries and the server sees the request twice. The byte stream did its job. The request needs its own identity.

A loss in a window.

1. Send bytes 1 through 8

2. Bytes 4 are lost

3. The receiver has 1 through 3

4. Retransmit from 4, or only 4 if the ack is selective

Window of four segments

1. Segments 1, 2, 3, 4 are in flight. Segment 2 is lost. The receiver buffers 3 and 4 if it is allowed, and acknowledges 1.

2. Duplicate acknowledgements of 1 tell the sender that later segments arrived and 2 did not, if you use that signal. A timeout would also fire.

3. Retransmitting only 2, then continuing, is what selective information allows. Retransmitting 2, 3, and 4 also works and wastes the link.

Problem 1. The byte stream delivers a write exactly once, the response packet is lost, and the client retries the request. How many times does the server perform the operation?

Answer. Twice, unless the operation is idempotent or the request carries an identifier the server has seen. Reliability of the byte stream is not exactly-once application semantics.

Problem 2. Why is a timeout of one second wrong on a path whose round-trip is two seconds, even when nothing is lost?

Answer. Every segment looks lost. You retransmit continuously and may inflate the congestion window's loss signals. The timeout has to sit above the real round-trip, including its variance.

Lecture 11 · 180 minutes

The secure channel

A byte stream can be read by anyone on the path. A secure channel adds confidentiality, integrity, and an identity. The handshake agrees on keys and checks a certificate. The record layer then encrypts and authenticates the bytes. The details of the ciphers belong to the security course. The networking obligation is to know what the handshake must finish before any application byte is trusted.

A certificate binds a name to a public key, if you trust the issuer. The client must check the name against the name it meant to reach. A valid certificate for a different name is a failure. You already saw that case. Here it sits in the connection setup, before HTTP.

Downgrade is a network attack on the handshake: the path strips the strong options and the endpoints accept a weak suite. A modern handshake refuses the weak suite rather than being helpful. Helpful, in this layer, is how you end up unencrypted.

The channel ends at the process that holds the key. A log line that prints the decrypted request is fine inside the server and a breach if the log is the path. Know where the channel stops.

Before the first request.

TCP connects -> Handshake and certificate check -> Keys exist -> Only then an application byte

A certificate for the wrong host

1. The user asked for mail.example. The certificate is valid, unexpired, and names files.example.

2. The signature check passes. The name check fails. A client that only checks the signature will send the user's password to the wrong host.

3. The connection must stop before the request. There is no later HTTP status that repairs a handshake with the wrong party.

Problem 1. The handshake completes and the application then sends a session cookie in the clear on a second connection. What did the secure channel protect?

Answer. Only the connection that used it. The cookie on the clear connection is visible to the path. The channel does not bless the application. Every connection that carries the secret has to be inside a channel.

Problem 2. Why does a handshake that accepts 'no encryption' as an option fail even if both honest endpoints prefer encryption?

Answer. The path can strip the preference and leave both sides believing the other offered nothing. If the protocol allows the null suite, the attacker selects it. Refuse the null suite.

Lecture 12 · 180 minutes

What the application may assume

Write down the assumptions a careful application is allowed to make. Bytes on one connection arrive in order, or the connection fails. They are not secret and not authentic unless a secure channel said so. A request may be performed more than once if the client retries. A name may resolve to a stale address until the time-to-live expires. A send rate that ignores loss will harm the other flows and may be policed.

Write down what it may not assume. A fixed latency. A path that stays up for the whole transfer. An address that means a particular machine next week. A queue that is fair to a flow that will not mark itself. Delivery of a datagram. If you need those, you build them, and you can, because the lower layers told you the truth about what they withhold.

The programming labs in a networks course are the test of this list. A byte-stream implementation that loses a sequence number will violate the first assumption. You should be able to point at the line.

The hours for this course include computing one window, one AIMD step, and one count-to-infinity by hand. The notes are the method. The arithmetic is the evidence you did it.

The contract.

You may assume

In-order bytes or a clean failure

A checked name on a secure channel

Loss as a congestion signal

You may not

A stable address

Exactly-once side effects

A fair share if you ignore loss

A retry that double-charges

1. The client sends 'pay.' The server commits the payment and the response is lost.

2. The client times out and sends 'pay' again. The server commits a second payment.

3. A request id stored with the payment lets the server recognize the retry and return the first result. The byte stream could not do that for you. The id is the application's sequence number.

Problem 1. List one bug that breaks in-order delivery and one bug that breaks exactly-once payment even if delivery is perfect.

Answer. Reordering sequence numbers, or acknowledging a byte you did not receive, breaks order. A retry after a lost response breaks exactly-once payment even when every byte of each attempt arrives in order. They are different layers.

Problem 2. DNS returns an old address and the connection fails. Which layer is wrong, and which layer is working?

Answer. The name layer is stale, within its time-to-live. Routing may be delivering to the address you asked for. Debug the resolver cache before you debug the routes.

Lecture 13 · 180 minutes

Measuring a path

A path has a delay, a loss rate, and a bottleneck rate. Ping measures a round-trip for a small packet, which is not the transfer time of a large file and not a one-way delay. A throughput test fills the window and measures bytes over time. Mixing the two numbers up is how people conclude a path is broken when it is merely long.

The bandwidth-delay product is the bottleneck rate times the round-trip time. It is the amount of data that must be in flight to keep the pipe full. A window smaller than that product leaves the link idle. A window much larger fills a buffer and adds delay. You computed a version of this in the first congestion lecture. Here you measure the inputs instead of being handed them.

Loss during a short test may be the probe itself. A test that opens many connections will induce the congestion it reports. One connection, a stated duration, and the window's behavior are the measurement.

Write the topology you think you measured. A number without the path is not a result. Wireless, a VPN, and a congested home router are different paths that can share an address.

Three different numbers.

Round-trip of a small packet

Bottleneck rate

Their product, the window you need

A long fat pipe

1. Rate 1 gigabit per second, round-trip 100 milliseconds. The product is 100 megabits, about 12.5 megabytes in flight.

2. A window of 64 kilobytes fills 64/12500 of the pipe. The transfer is window-limited and the link looks slow.

3. Raising the window to the product, if the path can buffer the fair share, raises the throughput. Raising it far past the product raises delay. Measure the round-trip as you raise it.

Problem 1. Ping takes 20 milliseconds and a file transfer is slow. Why is that not a contradiction?

Answer. Ping did not fill the pipe. The transfer may be window-limited, lossy, or sharing the link. The small-packet delay can be fine while the throughput is not. Measure the throughput and the window separately.

Problem 2. You open 32 parallel connections to 'measure the path' and see loss. What did you measure?

Answer. The path with your 32 connections on it. You may have created the congestion. A single connection's loss and window tell you more about the path you will actually use.

Lecture 14 · 180 minutes

A protocol you could implement

The closing project is a small reliable protocol over an unreliable datagram, or a careful client of a byte stream that retries without double effects. Either one forces the contract into code. Pick one.

If you build the reliable layer, the tests are: loss of a data segment, loss of an acknowledgement, reordering, and a timeout that is shorter than the round-trip. The last one should not cause a failure of correctness, though it will waste the link. If your protocol breaks when an acknowledgement arrives late, the timeout handling is wrong.

If you build the client, the tests are: a lost response, a stale DNS name you simulate with a bad address, and a status code your parser must not treat as success. Write the tests first. The software-engineering course was about that order for a reason.

Document the invariant in a paragraph. Sequence numbers increase. A byte is not delivered twice. A retry carries an id. Whichever you chose, the paragraph should be falsifiable by one of the tests.

Tests before the happy path.

1. Loss of data

2. Loss of an acknowledgement

3. Reorder

4. A timeout shorter than the round-trip

A late acknowledgement

1. You send segment 5 and time out. You send it again. Then both acknowledgements arrive.

2. The sender must ignore the duplicate acknowledgement or treat it as already satisfied, and it must not advance the window twice.

3. The receiver must ignore the duplicate segment or deliver it only once. Both sides need a memory of what they have already accepted.

Problem 1. A test drops every other acknowledgement and your sender stalls forever. What state is missing?

Answer. A retransmission timer, or a reading of duplicate information that would trigger a resend. Reliability requires a way to notice the absence of an acknowledgement. A sender that only advances on the first ack and never times out will stall as soon as one ack is lost.

Problem 2. Your client retries a POST when the connection resets after the server committed. What must the server store?

Answer. An idempotency key or request id, and the result of the first commit, so the retry returns that result instead of committing again. The reset is not evidence that the commit did not happen.